



第4章:中级SQL

数据库系统概念, 6th编辑。

©Silberschatz, Korth和Sudarshan参见www.db-book.com了解
重用条件



第4章:Intermediate SQL

“Join表达式”

☒视图

☒事务

完整性约束

SQL数据类型和模式授权



加入了关系

联合操作接受两个关系并返回另一个关系作为结果。

“连接操作是一个笛卡尔积，它要求两个关系中的元组匹配(在某些条件下)。它还指定了在连接结果中存在的属性。

连接操作通常用作from子句中的子查询表达式。



Join operations –Example

“关系过程”

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

“关系前置”

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

注:CS-315缺少prereq信息, CS-437缺少课程信息。



外连接

避免信息丢失的连接操作的扩展。“计算连接，然后将一个关系中不匹配另一个关系中的元组的元组添加到连接的结果中。”

使用空值。



Left Outer Join

☒ *Course* 自然左外连接 *prereq*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>

注:将*prere_id*读作*prereq_id*



Right Outer Join

“**course** **natural right outer join** *preeq* .

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101



Full Outer Join

◆课程自然全外连接 preeq

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101



加入了关系

联合操作接受两个关系并返回另一个关系作为结果。

这些额外的操作通常用作from子句中的子查询表达式

“**连接条件**”-定义两个关系中哪个元组匹配，以及在连接结果中存在哪些属性。

“**连接类型**”-定义如何处理每个关系中不匹配另一个关系中的任何元组的元组(基于连接条件)。

<i>Join types</i>
inner join
left outer join
right outer join
full outer join

<i>Join Conditions</i>
natural
on <predicate>
using ($A_1, A_1, \dots, A_n$)



连接关系-示例

course 内部连接 **prereq** on *course.course_id = prereq.course_id*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>	<i>course_id</i>
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190

⊗ **course left outer join prereq** on *course.course_id = prereq.course_id*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>	<i>course_id</i>
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190
CS-315	Robotics	Comp. Sci.	3	<i>null</i>	<i>null</i>



加入关系-示例

course natural right outer join prereq

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101

⊗ **course right outer join prereq using** (*course_id*)

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prere_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	<i>null</i>
CS-347	<i>null</i>	<i>null</i>	<i>null</i>	CS-101



V意见

在某些情况下，不希望所有用户都能看到整个逻辑模型(即数据库中存储的所有实际关系)。

“考虑一个需要知道讲师姓名和部门，但不知道薪水的人。这个人应该看到一个关系式，用SQL描述为

```
select ID, name, dept_name  
从教练
```

“视图提供了一种机制，可以在某些用户的视图中隐藏某些数据。

“任何不属于概念模型但作为“虚拟关系”对用户可见的关系都称为视图。



V 背影定义

视图是用**create view**语句定义的，它的形式是

创建视图v为<查询表达式>

其中<查询表达式>是任何合法的SQL表达式。视图名用v表示。

- ☒ 一旦定义了视图，视图名就可以用来引用视图生成的虚拟关系。
- ☒ 视图定义与通过求值查询表达式来创建一个新的关系是不一样的
 - z 更确切地说，视图定义导致保存表达式；
表达式被替换成使用视图的查询。



例子的观点

“没有薪水的教师视图

创建view *faculty* as

select *ID, name, dept_name*

从教练

*查找生物系所有讲师选择姓名

从教师

where *dept_name* = ' Biology '

创建部门工资总额的视图

创建视图 *departments_total_salary*(*dept_name, total_salary*) 为 **select** *dept_*

name, sum (*salary*)

从教练

组由 *dept_name*;



V使用其他视图定义的视图

“**create view** *physics_fall_2009* **as** .
选择课程。Course_id, sec_id, building, room_number **from** course, section
在课程。Course_id = section.course_id和course.dept_name = '
Physics '
和部分。学期= '秋天'
和部分。年= '2009' ;

“**create view** *physics_fall_2009_watson* **as** **select** course_id,
room_number **from** *physics_fall_2009* .
where building= ' Watson ';



V 背影扩张

在查询/另一个视图中扩展视图的使用

创建视图 *physics_fall_2009_watson*

(select *course_id*, *room_number* .

From (select *course.*) *Course_id*, *building*, *room_number* **from** *course*,
section

在课程。*Course_id* = *section.course_id*和*course.dept_name* = '

Physics '

和部分。*学期* = '秋天'

和部分。*年* = '2009')

where *building* = ' Watson ';



v使用其他视图定义的视图

一个视图可以在定义另一个视图的表达式中使用，如果在定义 v_1 的表达式中使用 v_2 ，则认为视图关系 v_1 直接依赖于视图关系

v_2

如果 v_1 直接依赖于 v_2 或者存在从 v_1 到 v_2 的依赖路径，则视图关系 v_1 被称为依赖于视图关系 v_2

如果视图关系 v 依赖于自身，则称其为递归关系。



V 背影扩张

一种定义用其他视图定义的视图的含义的方法。让视图 v_1 由表达式 e_1 定义，该表达式本身可能包含视图关系的使用。

一个表达式的视图扩展重复下面的替换
步骤：

重复

在 e_1 中找到任意视图关系 v_i

用定义 v_i 的表达式替换视图关系 v_i ，直到 e_1 中不再存

在视图关系

“只要视图定义不是递归的，这个循环就会终止。”



View的更新

- ☒ 向我们之前定义的教员视图添加一个新的元组
insert into教职工值(' 30765 ', ' Green ', ' Music ');这个插入必须用元组的插入来表示

(' 30765 ', ' Green ', ' Music ', null)

成指导员关系。



有些更新不能唯一翻译

“**create view** *instructor_info* **as** .

选择ID,名称,建筑

来自教员,系

其中 *instructor.dept_name = department.dept_name*;

' **insert into** 教练员信息值(' 69987 ', ' White ', ' 泰勒');如果泰勒
有多个部门, 是哪个部门?

如果泰勒没有部门怎么办?

大多数SQL实现只允许对简单视图进行更新, **from**子句只有一个数据库关系。

select子句只包含关系的属性名, 没有任何表达式、聚合或不同的规范。

任何未在**select**子句中列出的属性都可以设置为null

查询中没有**group by**或**having**子句。



和Some Not at All

“**create view** *history_instructor as*”

select *

从教练

where *dept_name*= ' History ';

' Insert (' 25566 ', ' Brown ', ' Biology ', 100000) into *history_instructor* .



交易

- 工作单位
原子事务

- z 完全执行或回滚，就好像它从未发生过一样

- “交易隐式开始”

- z 以提交工作或回滚工作结束

但是大多数数据库默认:每个SQL语句自动提交

在SQL:1999中，可以使用:**begin atomic**结束



Integrity约束

完整性约束通过确保对数据库的授权更改不会导致数据一致性的丢失来防止对数据库的意外损坏。

z 支票账户必须有大于\$10,000.00的余额。

银行职员的工资必须至少是每小时4美元。

客户必须有一个(非空的)电话号码。



单个关系上的约束

* **not null**

* **主键**

☒ **独特**

check (P), 其中P是谓词



非空和唯一约束

* not null

z 声明名称和预算不为空

Name **varchar(20)**不为空

预算数字(12,2)不空

☒ **unique** ($A_1, A_2, \dots, A_m$)

z unique规范说明属性 $A_1, A_2, \dots, A_m$ 构成一个候选键。

z 候选键允许为空(与主键相反)。



check子句

“检查(P)”

其中P是谓词

例如:确保学期是秋季、冬季、春季或夏季之一:

创建table *section* (

Course_id **varchar** (8),

Sec_id **varchar** (8),

学期管理员(6);

年数值(4,0),

Building **varchar** (15),

Room_number **varchar** (7),

时隙id **varchar** (4),

主 键 (course_id, sec_id, semester, year), **check**(学 期 in(‘
Fall’ , ‘Winter’ , ‘Spring’ , ‘Summer’))

);



参照完整性

确保对于给定的一组属性出现在一个关系中的值也出现在另一个关系中的一组属性中。

z示例:如果“生物学”是出现在教师关系中的一个元组中的院系名称, 则有
在部门关系中存在一个“生物学”的元组。设A为一组属性。设R和S是两个包含属性A的关系, 其中A是S的主键, 如果对于出现在R中的A的任何值, 这些值也出现在S中, 则称A是R的**外键**。



参照完整性中的级联动作

```
create table course ()  
    Course_id char(5)主键,  
    标题 varchar (20),  
    Dept_name varchar(20)引用部门)
```

```
* create table course (  
    ...  
    dept_name varchar (20),  
    外键(dept_name)在删除级联时引用department  
        在更新级联上,  
    ...  
)
```

“级联的替代操作:设置null, 设置默认



I交易期间的完整性约束违反

例如,



创建table *person* ()

ID char (10),

名字char (40),

母亲char (10),

父亲char (10),

主键*ID*;

外键父引用*person*, 外键母引用*person*)



如何插入元组?



如果母亲或父亲被声明为非空怎么办?

*z*约束*father_ref*外键*father*引用*person*,

约束*mother_ref*外键 父引用 *person*) *z* set constraints *father_ref*,

mother_ref deferred



复杂的Check子句

“**check** (time_slot_id **in** (select *time_slot_id* **from** *time_slot*)) *z*

为什么不在这里使用外键呢?”

每个section至少有一个教员在教授这个section。 *z*这个怎么写?

“不幸的是:check子句中的子查询几乎不被任何数据库支持

*z*备选方案:触发器(稍后)

“**create assertion** <assert -name> **check** <**predicate**>;

*z*也不被任何人支持



Built-in Data Types in SQL

“**date**”: 日期, 包含(4位)年、月和日期 z 示例: **日期** '2005-7-27'

“**time**”: 一天中的时间, 以小时、分钟和秒为单位。

z 示例: **time** '09:00:30' **time** '09:00:30.75'

timestamp: 日期加上一天中的时间

z 示例: **时间戳** '2005-7-27 09:00:30.75'

“**interval**”: 时间段

z 示例: 间隔 “1” 天

z 从另一个中减去一个日期/时间/时间戳值得到一个间隔值

z 间隔值可以加到日期/时间/时间戳值中



其他功能

“**create table student**”

(ID **varchar** (5),
Name **varchar**(20)不空,
Dept_name **varchar** (20),
tot_cred **numeric** (3,0) **default** 0,
主键(ID))

*创建索引|studententid索引/学生(ID) *大对象

z书评clob(10KB)

z图像blob(10MB)

z电影blob(2GB)



User-Defined Types

- ☒ SQL中的**create type** construct创建用户定义的类型

创建类型美元为数字(12,2)final

创建表 *department* (dept_name
 varchar (20), *building*
 varchar(15), 预算美元);



域

在SQL-92中**创建域**构造创建用户定义的域类型

创建域*person_name* **char(20)**不为空

类型和领域是相似的。域可以在其上指定约束，例如不为空。

»**create domain** *degree_level* **varchar(10)**

约束*degree_level_test*

check (value **in** (' bachelor ', ' Masters ', ' phd '));



大对象类型

“大对象(照片、视频、CAD文件等)以大对象的形式存储:

- z **blob**:二进制大对象——对象是大量未解释的二进制数据的集合(其解释留给数据库系统外的应用程序)

- z **blob**:字符大对象——对象是字符数据的大集合

- z 查询返回大对象时, 返回的是指针而不是大对象本身。



授权

部分数据库的授权形式:

Read -允许读取，但不允许修改数据。

“插入” -允许插入新数据，但不允许修改现有数据。 **更新**-允许修改数据，但不允许删除数据。

“删除” -允许删除数据。

修改数据库模式的授权形式

索引-允许创建和删除索引。

“资源” ——允许创建新的关系。

改变——允许添加或删除关系中的属性。 **“删除”** ——允许删除关系。



SQL中的授权规范

grant语句用于授予授权**grant** <特权列表>

在<关系名或视图名>上授予<用户列表>

<用户列表>为:

z a user-id

z **public**, 它允许所有有效用户获得授予的特权

z A角色(后面会详细介绍)

在视图上授予特权并不意味着在底层关系上授予任何特权。

特权的授予者必须已经拥有对指定项的特权(或者是数据库管理员)。



SQL中的特权

select: 允许读访问关系，或使用视图查询的能力

z 示例: 授予用户 U_1 、 U_2 和 U_3 选择分支关系上的授权:

grant select on instructor to U_1 , U_2 , U_3

insert: 插入元组的能力。

update: 使用SQL更新语句进行更新的能力。 **delete:** 删除元组的能力。

所有特权: 用作所有允许的特权的缩写形式。



撤销SQL中的授权

“**revoke**”语句用于撤销授权。 **Revoke** <特权列表>

从<用户列表> <关系名或视图名>

☒例子:

revoke select on branch from U_1, U_2, U_3

<privilege-list>可以是all，用来**撤销被**撤销者可能拥有的所有特权。

如果<revokee-list>包含public，则所有用户都失去特权，除了那些显式授予它的用户。

如果同一用户被不同的授予者授予两次相同的权限，则该用户在撤销后可以保留该权限。

所有依赖于被撤销特权的特权也会被撤销。



角色

“**创设角色**指导者;
可授予角色特权:

z授予对讲师的选择权限;
角色可以授予用户, 也可以授予其他角色**z****创建角色***student*

z将*instructor*授予Amit;

z创设角色院长;

z授予*dean*讲师;

z授予*dean*给中本聪;



视图授权

```
create view geo_instructor as
(select * .
  从教练
  where dept_name = '地质学');
“grant select on geo_instructor to staff” 假
设工作人员发出z select *
  从geo_instructor;
“如果……
```

z员工对讲师没有权限?

视图的z创建者对指导员没有一些权限?



其他授权功能

引用特权来创建外键

授予部门上的引用(dept_name)给Mariano;**z**为什么需要这个?

“转让特权”

向Amit**授予**一个部门, 并提供授予选项;**撤销**Amit、Satoshi级联中的**select on department**;

撤销Amit的**select on department**, Satoshi**限制**;**等**, 请参阅第4.6节, 了解此处省略的更多细节。



第4章结束

数据库系统概念，6th编辑。

©Silberschatz, Korth和Sudarshan参见www.db-book.com了解
重用条件



Figure 4.01

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>tot_cred</i>
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120



Figure 4.02

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>grade</i>
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	<i>null</i>



Figure 4.03

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>tot_cred</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>grade</i>
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2009	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2009	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2009	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2009	A
12345	Shankar	History	32	CS-315	1	Spring	2010	A
12345	Shankar	Finance	32	CS-347	1	Fall	2009	A
19991	Brandt	Music	80	HIS-351	1	Spring	2010	B
23121	Chavez	Physics	110	FIN-201	1	Spring	2010	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2009	B-
45678	Levy	Physics	46	CS-101	1	Fall	2009	F
45678	Levy	Physics	46	CS-101	1	Spring	2010	B+
45678	Levy	Physics	46	CS-319	1	Spring	2010	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2009	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2009	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2010	A-
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2009	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2010	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2009	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2009	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2010	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2009	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2010	<i>null</i>



Figure 4.04

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>tot_cred</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>grade</i>
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2009	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2009	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2009	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2009	A
12345	Shankar	History	32	CS-315	1	Spring	2010	A
12345	Shankar	Finance	32	CS-347	1	Fall	2009	A
19991	Brandt	Music	80	HIS-351	1	Spring	2010	B
23121	Chavez	Physics	110	FIN-201	1	Spring	2010	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2009	B-
45678	Levy	Physics	46	CS-101	1	Fall	2009	F
45678	Levy	Physics	46	CS-101	1	Spring	2010	B+
45678	Levy	Physics	46	CS-319	1	Spring	2010	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2009	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2009	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2010	A-
70557	Snow	Physics	0	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2009	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2010	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2009	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2009	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2010	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2009	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2010	<i>null</i>



Figure 4.05

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>grade</i>	<i>name</i>	<i>dept_name</i>	<i>tot_cred</i>
00128	CS-101	1	Fall	2009	A	Zhang	Comp. Sci.	102
00128	CS-347	1	Fall	2009	A-	Zhang	Comp. Sci.	102
12345	CS-101	1	Fall	2009	C	Shankar	Comp. Sci.	32
12345	CS-190	2	Spring	2009	A	Shankar	Comp. Sci.	32
12345	CS-315	1	Spring	2010	A	Shankar	History	32
12345	CS-347	1	Fall	2009	A	Shankar	Finance	32
19991	HIS-351	1	Spring	2010	B	Brandt	Music	80
23121	FIN-201	1	Spring	2010	C+	Chavez	Physics	110
44553	PHY-101	1	Fall	2009	B-	Peltier	Physics	56
45678	CS-101	1	Fall	2009	F	Levy	Physics	46
45678	CS-101	1	Spring	2010	B+	Levy	Physics	46
45678	CS-319	1	Spring	2010	B	Levy	Physics	46
54321	CS-101	1	Fall	2009	A-	Williams	Comp. Sci.	54
54321	CS-190	2	Spring	2009	B+	Williams	Comp. Sci.	54
55739	MU-199	1	Spring	2010	A-	Sanchez	Music	38
70557	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	Snow	Physics	0
76543	CS-101	1	Fall	2009	A	Brown	Comp. Sci.	58
76543	CS-319	2	Spring	2010	A	Brown	Comp. Sci.	58
76653	EE-181	1	Spring	2009	C	Aoi	Elec. Eng.	60
98765	CS-101	1	Fall	2009	C-	Bourikas	Elec. Eng.	98
98765	CS-315	1	Spring	2010	B	Bourikas	Elec. Eng.	98
98988	BIO-101	1	Summer	2009	A	Tanaka	Biology	120
98988	BIO-301	1	Summer	2010	<i>null</i>	Tanaka	Biology	120



Figure 4.07

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
69987	White	<i>null</i>	<i>null</i>

instructor

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000
<i>null</i>	Painter	<i>null</i>

department



Figure 4.06

<i>Join types</i>
inner join
left outer join
right outer join
full outer join

<i>Join conditions</i>
natural
on <predicate>
using ($A_1, A_2, \dots, A_n$)



Figure 4.03

